

Synthetic Data Generation Is More Than Generating Random Rows

THE NEED

Organizations need realistic datasets for ML development, testing, analytics, software development and research.

THE COMPLICATION

- Sensitive information & privacy risk
- Missing values and quality issues
- Complex inter-column relationships
- Imbalanced distributions

MANUAL DECISION BURDEN

Traditional workflows require manual choices for schema, generation method, parameters, quality evaluation and regeneration.

TODAY: MANUAL PIPELINE



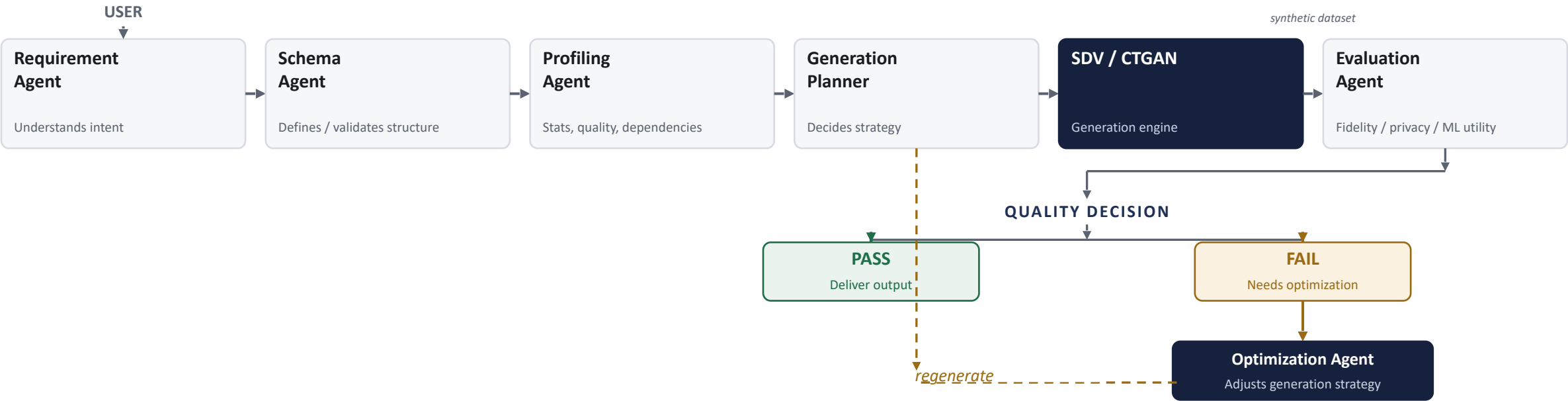
OUR SYSTEM: AUTONOMOUS LOOP



Problem Statement: Generating useful synthetic data requires multiple analysis, generation and evaluation decisions that are difficult to perform manually and consistently.

Objective: Build an autonomous agentic system that analyzes requirements/data, plans synthetic-data generation, evaluates the result and improves it when quality targets are not met.

Multi-Agent Architecture



AGENTS — ONE JOB EACH

Requirement Agent

Converts requirements into structured spec.

Generation Agent

Invokes the chosen generation engine.

Schema Agent

Creates / validates dataset schema & metadata.

Evaluation Agent

Checks fidelity, privacy risk, ML utility.

Profiling Agent

Analyzes distributions, quality, relationships.

Optimization Agent

Adjusts strategy after a failed evaluation.

Generation Planner

Decides the generation strategy.

LLM = Reasoning · LangGraph = Orchestration · Tools / Engines = Execution

Technology Stack & Responsibilities

L1	Reasoning	LLM: [Selected model / API] Understand requirements • Interpret profiling • Produce generation plans • Explain results • Decide on optimization
L2	Agent Orchestration	LangGraph State management • Agent-to-agent transitions • Conditional routing • Evaluation → optimization loop
L3	Data & Generation	Pandas · SDV (Gaussian Copula, CTGAN) · Faker Loading & profiling • Synthetic generation, metadata, constraints • Realistic individual values
L4	Evaluation	SDMetrics · scikit-learn Fidelity & similarity, privacy signals • ML utility (TSTR / TRTS experiments)
L5	Interface	Django Dataset upload • Requirement input • Generation controls • Results & download

LLM decides & reasons · SDV / CTGAN generate synthetic data · SDMetrics / scikit-learn evaluate · LangGraph orchestrates

Concrete Example: From Requirement to Synthetic Dataset

USER INPUT

“Generate 50,000 customer records for ML testing. Include customer_id, age, gender, income, city and credit_score. Age should be between 18 and 70. Preserve realistic relationships between age, income and credit score. Keep gender distribution reasonably balanced.”

SYSTEM PROCESS

- Requirement Agent
- Schema Agent
- Profiling / Relationship Analysis
- Generation Planner
- SDV + CTGAN
- Evaluation Agent → Quality Decision

EXPECTED SYNTHETIC OUTPUT (sample rows)

customer_id	age	gender	income	city	credit_score
C10001	28	Female	48,000	Jaipur	712
C10002	42	Male	76,000	Pune	748
C10003	35	Female	62,000	Delhi	731
C10004	51	Male	91,000	Mumbai	782

EXPECTED EVALUATION REPORT

```
Rows generated      : 50,000
Schema compliance   : PASS
Constraint validation : PASS
Fidelity            : 0.XX*
Privacy risk        : LOW / ACCEPTABLE*
ML Utility          : 0.XX*
Overall decision     : PASS / REGENERATE
```

**Placeholder values — actual experiments not yet performed. Privacy is evaluated as a risk/objective, not claimed as absolute.*

Expected output = **synthetic records + evaluation report + quality decision** — not just a data generator.

Risks and Assumptions

RISKS

- **Poor automatic schema inference**
Incorrect data types or semantic meaning could lead to poor generation.
- **Synthetic data quality**
Generated data may fail to preserve important distributions or relationships.
- **Model selection**
A single synthesizer may not work equally well for every dataset.
- **Privacy**
Synthetic data can still expose memorization or disclosure risk.
- **LLM decisions**
The LLM may produce invalid or unsupported generation plans.
- **Generation cost / runtime**
CTGAN-style models can require significant training time.

ASSUMPTIONS

- **Tabular scope**
Initial MVP focuses on tabular datasets.
- **Input format**
CSV is the primary supported input format initially.
- **Engine compatibility**
Generation engines must support the detected schema.
- **Configurable evaluation**
Evaluation thresholds will be configurable.

Current stage: Architecture & Planning → Implementation → Evaluation